

Neima Bravo

GitHub: nbravo.dev@outlook.com | UTEP: nbravo@miners.utep.edu

Submitted on September 21, 2026

CS 4475 – Operating Systems Concepts - Fall 2026

Dr. Shirley Moore

- Lab Report –

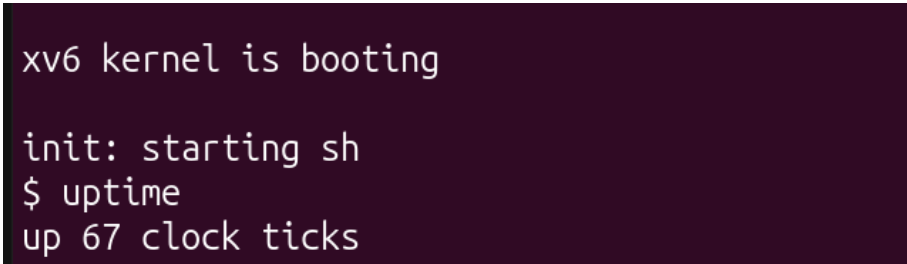
PA 1: Implementing xv6 User Commands and System Calls

Repository: <https://github.com/datanbravo/xv6-riscv>

Branch: pal-syscalls

Task 1. (5 pts) Implement the uptime utility.

For Task 1, I created user/uptime.c and used the existing uptime() system call to get the number of clock ticks since xv6 started. I printed that value and added uptime to UPROGS in the Makefile so xv6 would build the command. When I tested it, the command returned up 67 clock ticks.

A terminal window with a dark purple background and light green text. The text shows the xv6 kernel booting, the init process starting a shell, and the uptime command being executed, which returns 'up 67 clock ticks'.

```
xv6 kernel is booting
init: starting sh
$ uptime
up 67 clock ticks
```

Figure 1. Output from the uptime command in xv6.

Task 2. (10 pts) Implement a `time1` command that reports elapsed time.

For Task 2, I created `user/time1.c` and added `time1` to `UPROGS` in the `Makefile`. The program records uptime before `fork()`, runs the requested command in the child with `exec()`, and makes the parent wait for it to finish before recording uptime again. The difference between the two values is printed as the elapsed time.

I tested the no-command case, `time1 sleep 10`, and `time1 matmul`. The tests showed that `time1` waits for the command to finish and then reports the elapsed time in clock ticks.

```
$ time1
usage: time1 command
$ time1 sleep 10
elapsed time: 10 ticks
$ time1 matmul
Time: 7 ticks
elapsed time: 7 ticks
```

Figure 2. Output from the `time1` command.

Task 3. (10 pts) Keep track of how much `cputime` a process has used.

For Task 3, I changed three kernel files to track CPU time for each process. In `kernel/proc.h`, I added a `uint cputime` field to `struct proc`; in `kernel/proc.c`, I initialized it to 0 in `allocproc()`; and in `kernel/trap.c`, I incremented it on timer interrupts before the process yields the CPU. After fixing the timer-interrupt blocks, `xv6` compiled and booted normally.

```
nrbravo@buntu:~/xv6-riscv$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv/ -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffrees
tanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin
-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -f
no-pie -no-pie -c -o kernel/trap.o kernel/trap.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/kalloc.o kernel/spinlock.o kernel/s
tring.o kernel/main.o kernel/vm.o kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o
kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RMX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ / / /; /$/' > kernel/kernel.sym
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device
,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$
```

Figure 3. `make qemu` succeeded.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

For Task 4, I created kernel/pstat.h with struct rusage, then added the wait2() declaration and user entry in user/user.h and user/usys.pl. I registered the system call in kernel/syscall.h and kernel/syscall.c, added kwait2() to kernel/defs.h, and connected sys_wait2() in kernel/sysproc.c. In kernel/proc.c, I used the existing kwait() logic as the base for kwait2() and added the code needed to return the child's cputime before the process is freed.

```
.o user/sync.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_sync user/sync.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_sync > user/sync.asm
riscv64-unknown-elf-objdump -t user/_sync | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/sync.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/uptime.o user/uptime.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_uptime user/uptime.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_uptime > user/uptime.asm
riscv64-unknown-elf-objdump -t user/_uptime | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/uptime.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/matmul.o user/matmul.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_matmul user/matmul.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_matmul > user/matmul.asm
riscv64-unknown-elf-objdump -t user/_matmul | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/matmul.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/sleep.o user/sleep.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/sleep.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_sleep > user/sleep.asm
riscv64-unknown-elf-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/sleep.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/time1.o user/time1.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_time1 user/time1.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_time1 > user/time1.asm
riscv64-unknown-elf-objdump -t user/_time1 | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/time1.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_uptime user/_matmul user/_sleep user/_time1
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1120 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$
```

Figure 4. make qemu succeeded.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

For Task 5, I created user/time.c and added time to UPROGS in the Makefile. It is based on time1, but it uses wait2() to get CPU time along with elapsed time. It then calculates %CPU as (cputime * 100) / elapsed time, with a check to avoid dividing by zero.

I tested time with sleep and matmul. Sleep showed elapsed time with very little CPU time, while matmul showed a CPU-bound process using most of the CPU. Running two matmul commands at once also showed the processes sharing the single CPU.

```
nrbravo@Ubuntu:~/xv6-riscv$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/nrbravo/xv6-riscv/. -march=rv64gc -std=gnu99 -MD -mcmmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o user/time.o user/time.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_time user/time.o user/ulib.o user/usys.o user/printf.o user/unalloc.o
riscv64-unknown-elf-objdump -S user/_time > user/time.asm
riscv64-unknown-elf-objdump -t user/_time | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/time.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_uptime user/_matmul user/_sleep user/_time1 user/_time
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1157 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time
usage: time command
$ time sleep 10
elapsed time: 11 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul
Time: 6 ticks
elapsed time: 7 ticks, cpu time: 7 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 13 ticks
elapsed time: 14 ticks, cpu time: 7 ticks, 50% CPU
Time: 12 ticks
elapsed time: 13 ticks, cpu time: 7 ticks, 53% CPU
$
```

Figure 5. Output from the time command.

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment helped me understand how user commands connect to the xv6 kernel and how the different process functions work together. The main problems I ran into were the Linux workflow failing because clang-format was missing and a kerneltrap panic in Task 3 because I forgot braces around the timer-interrupt code. I fixed them by installing clang-format, using make fmt before pushing, and correcting the if blocks before rebuilding xv6.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One limitation is that the time command only reports the CPU time of the direct child that wait2() waits for. If that command creates more child processes, their CPU time is not added to the original child's value, so the result may not show the total CPU used by the whole command. It also uses clock ticks and integer percentages, so it is less precise than the full Linux time utility and does not report separate user time, kernel time, or memory usage.

Resources / AI Use

I used ChatGPT as a tutoring and debugging resource while completing the assignment. It helped me interpret the task requirements, review my time1 implementation, understand the files needed for wait2(), and troubleshoot errors such as the missing clang-format installation and the kerneltrap caused by missing braces in Task 3. I used the explanations as guidance, made the changes in my Ubuntu VirtualBox VM, tested them with make fmt and make qemu, and committed and pushed the final code myself.